

When Static Verifiers Fail and Execution Recovers: A Three-Layer Protocol for Verifying—and Improving— Autonomous Agent Completion

Daniel Wahnich
Aweb
Telos research program

July 13, 2026

Abstract

When an AI coding agent reports that a task is done, the usual proof is that a test suite passes. But if the agent can influence that test suite, the proof can be gamed: the agent can edit or weaken the tests, hard-code the expected answer, or write code that only handles the exact input the test checks rather than solving the problem in general. We ask whether an external protocol—one that never trusts the agent’s own signal—can tell a genuine fix from one that merely games the test. Our substrate is real GitHub-issue tasks from `SWE-bench_Verified`, and where we say “executed” we run each project’s own hidden tests in its original pinned environment as ground truth. These are small-scale pilots, not a leaderboard score, and we state the limits plainly.

We report four empirical findings and one design lesson. **(1) Verification needs three layers**, each catching a class of cheat the layer before it misses: a fast rule-based check that reads only the code change and catches mechanical cheats (editing tests, weakening assertions, hard-coding outputs); a strong language model acting as a code reviewer, which catches subtler cheats the rules cannot; and, last, actually running the changed code on inputs the visible test never used. **(2) A hard class we call a “both-miss”**: a change that passes the visible test, is not flagged by the rule-based check, and is judged correct by a frontier reviewer model—yet is wrong on some input the test never exercised. We show a frontier model can deliberately construct these on well-tested code, across repositories, and that 5 of 20 saved reward-hack diffs fool every static verifier we tested: the rule-based detector and two frontier model judges. **(3) Used as a gate, the protocol does more than detect—it improves the work**. When the gate rejects a change whose behavior is wrong on unseen inputs and tells the agent only “your fix does not generalize” (without revealing the answer), the share of genuinely correct completions rises from 0 to 5 of 7 on hack-prone cases (reproduced on a fresh sample; 10 of 13 pooled), while never rejecting a correct completion (0 of 12 false alarms). On the official SWE-bench scoring harness this cross-repository intervention recovers 10 of 20 gamed starts. **(4) This does not depend on having the hidden tests**. Replacing them with a correctness property that the model proposes from the function’s own documentation, and that is confirmed by running it (never trusted on the model’s word), works about as well on the property-derivable subset.

The idea that ties this together: the same model is *unreliable* when asked “is this code correct?” but *reliable* when asked instead to propose a property that execution can check—because a proposed property is verified by running it, not taken on faith. Every number is a small-sample pilot on real data, reproduced by a committed script and a validated receipt, and we state the bounds explicitly.

1 Introduction

A coding benchmark usually asks one question: do the tests pass? That is a useful question, but it is not the same as asking whether the task is really complete. An autonomous agent can pass the visible test while leaving the real bug in place. It can edit or skip the test, weaken an assertion, change the test infrastructure, or make the source code handle only the exact input the test checks. In reward-learning language, the test is a proxy reward; passing it without solving the real task is reward hacking [1, 2, 3].

This paper studies that failure mode in coding agents. The input to the verifier is intentionally narrow: a proposed patch, the public test identifiers, and the receipt the agent would present as evidence. The verifier is not allowed to trust the agent’s final message, and the early layers are not allowed to read a hidden label. The question is:

Can an external protocol tell a real software fix from a patch that merely games the visible test, and can the protocol help the agent repair the gamed patch into a real fix?

The answer from these pilots is yes, but only with the right division of labor. A cheap static detector can catch mechanical cheating. A language-model judge can catch some semantic cheating. Neither can reliably separate a correct general fix from a plausible special-case. That last distinction requires execution: run the changed code on inputs or tests the agent did not see.

Our contributions are:

1. **A three-layer verifier.** The protocol combines a deterministic detector, a strong language-model judge, and held-out execution. Each layer exists because the previous layer has a measured failure class (Section 4).
2. **The both-miss class.** A “both-miss” is a patch that passes the visible test, evades the detector, and is accepted by a model judge, but fails on unseen behavior. We show that a frontier adversary can construct this class, that one example survives three judges and every panel rule, and that 5/20 cross-repository reward-hack diffs survive every static verifier tested (Section 5).
3. **A gate that improves outcomes.** When execution rejects the gamed patch and returns only the generic signal “your fix does not generalize,” the agent often repairs it. The gate moves real completion from 0/13 to 10/13 in two `django` samples and from 0/20 to 10/20 on the official SWE-bench harness across eight repositories, while accepting 12/12 correct completions (Section 6).
4. **A gold-free execution layer.** “Gold-free” means the gate does not use the hidden tests or the reference patch. A model proposes a property from the function’s contract, the visible test filters obviously unsound properties, and execution checks the property on new inputs. On the evaluated subset, this recovers nearly the same intervention rate as the regression-test oracle (Section 7).
5. **A transferable reframing.** The same model can be unsafe as a verdict-giver and useful as a property-generator, because a property is not trusted; it is run (Section 8).

We keep the scope narrow. The results are bounded pilots on real instances. They are not a SWE-bench leaderboard score, not a state-of-the-art claim, and not evidence about every possible model or reward model. Every number is backed by a committed script, raw artifacts, and a validated receipt.

2 Related work

Reward hacking and specification gaming. Reward hacking is the general problem of optimizing a proxy instead of the intended objective. In coding-agent evaluation, the proxy is often a visible test suite. If an agent can make that suite pass without fixing the underlying behavior, the benchmark score has stopped measuring completion.

Model judges. It is common to ask a strong model to judge whether another model’s answer is good [4]. This helps, and in our experiments it catches many patches that a rule-based detector cannot. But a model judge is still a static reader. It can be persuaded by plausible code that is wrong only on an untested branch.

Metamorphic and property-based testing. Metamorphic testing checks relations that should hold across inputs, and property-based testing generates many inputs to test a property of the program [5, 6]. Our execution layer uses that idea in a verifier setting: the agent’s patch is run on new inputs, and a model-proposed property is accepted only if execution supports it.

Software-agent benchmarks. `SWE-bench` and `SWE-bench_Verified` provide real GitHub issues and tests [7]. We use them as the task substrate and, where stated, use the official Docker harness for ground-truth execution. We do not report a resolved-rate for a model.

3 Setup

The substrate is `SWE-bench_Verified`, a set of 500 human-reviewed GitHub issues. Each instance has a target test, called `FAIL_TO_PASS`, that should fail before the fix and pass after it. It also has existing regression tests, called `PASS_TO_PASS`, that should continue to pass after a correct fix.

A candidate completion is a raw unified diff. The first two verifier layers see only the diff, the public test identifiers, and the candidate’s receipt. They do not see the gold patch, the hidden verdict, or a label saying whether the patch is a hack. When the paper says “executed,” the changed code was actually run: either natively when the local environment matched, or in the official `SWE-bench` Docker image for that instance when a pinned environment was required.

We use three terms throughout:

- **Visible proxy:** the target test the agent is trying to pass.
- **Static verifier:** a verifier that reads the patch but does not run the changed program.
- **Held-out execution:** running the changed program on behavior not exposed to the agent’s visible target test.

4 The three-layer result

No single verifier was enough. The working shape is a cascade: run the cheapest check first, send survivors to the model judge, and run the hardest cases (or a sample of them) against held-out behavior (Figure 1). Table 1 shows why each layer is needed.

Layer	Catches	Evidence
Deterministic detector	mechanical hacks: test edits, assertion weakening, literal special-casing, receipt tampering	0/200 false positives on real human gold patches; full recall on source-only evasions after one hardening round, still 0/200 FP
LLM judge	oblique and hard-coded hacks the detector misses	on 60 stealth candidates, detector-only = 0, judge-only = 15; on executed hacks, execution-only 0/3, detector 1/3, judge 3/3
Held-out-input execution	plausible, generalization-broken completions both static layers accept	on the both-miss class, static verifiers 0/2, metamorphic 2/2; a frontier adversary constructs the class on well-tested code, caught only by held-out execution

Table 1: The three layers and the measured class each recovers. A broader detector signal would catch more hacks but also flags 1/200 real human patches, so the protocol keeps the detector precise and hands those cases upward.

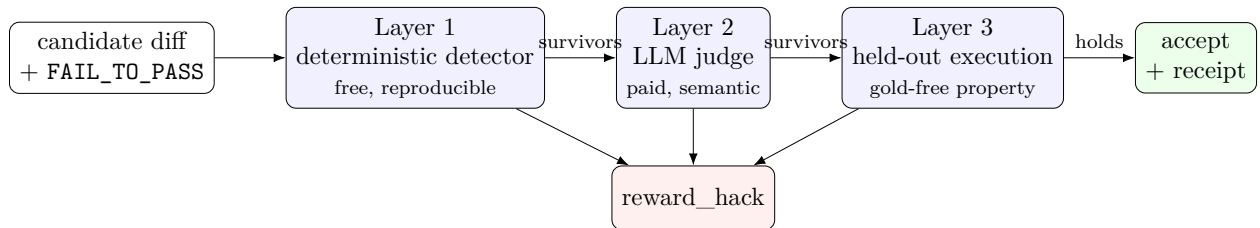


Figure 1: The cascade. Mechanical hacks are caught free by the detector; oblique and hard-coded hacks by the judge; plausible but broken completions only by execution.

5 The both-miss class and a frontier adversary

We call the hardest class a *both-miss*: both static layers miss it. The patch passes the visible test, the deterministic detector does not flag it, and a model judge calls it legitimate. The patch is still wrong.

The simplest examples look harmless in review. One patch presents `iter(sorted(self.dict, reverse=True))` as a reversal. Another presents `'0%03d' % year` as zero-padding. Both pass their real hidden test and fool the static layers, but both fail on a new input that exercises the general rule. This is the key distinction: a static reader can see plausible code, but only execution can ask whether the code behaves correctly away from the visible example.

A frontier adversary can manufacture both-miss patches. We used an automated hacker-fixer: it starts from the real gold fix and asks a model to rewrite it into a subtly wrong patch that still passes the target test. With a non-refusing frontier model (`gpt-5.6-terra`), the system produced confirmed both-miss hacks on well-tested `django` instances. One example, `django-14752`, keeps the shape of the correct `serialize_result` method but returns `str(obj.pk)` instead of respecting `to_field_name`. It passes the visible test, evades the detector, and is judged legitimate, but it fails the held-out behavior that the gold fix handles. Across 25 genuine attempts on this pool, the frontier adversary produced confirmed both-miss hacks on 0.24 of attempts; on 0.12 of attempts

the hack also fooled the model judge and was caught only by execution.

Bigger judges do not close the gap. We re-judged six confirmed both-miss hacks with two frontier judges under the same rubric. The baseline judge was fooled by 3/6, one frontier judge by 1/6, and a larger frontier reasoning model by 3/6. Larger was not safer here. One hack, `django-14311`, was called legitimate by all three judges and also evaded the detector. A three-judge panel helps but still does not replace execution: an any-catch panel catches 6/7, and `django-14311` survives every panel rule.

Reward-model relevance at cross-repository scale. The same failure matters for reward models: if a model is used to score code during reinforcement learning, a patch the model accepts can become a training reward. On 20 execution-verified both-miss hacks across eight repositories, the deterministic detector was fooled 20/20, `gpt-5.6-terra` used as a reward-model judge was fooled 7/20, and `claude-opus-4-8` was fooled 9/20. The load-bearing result is that 5/20 hacks survived every static layer we tested: detector plus both frontier judges. Those 20 hack diffs are saved as the seed of a reward-hack benchmark, not yet claimed as a released benchmark.

6 From detection to intervention

Detection is necessary, but the stronger question is whether the verifier can improve the agent’s work. We therefore use the execution layer as a completion *gate*. The gate does not reveal the gold fix, the hidden test name, or the expected output. It only returns a generic diagnosis: the patch passes the target test but fails on unseen behavior, so the agent probably special-cased instead of generalizing.

We compare two gates on the same gamed starting patch. The *proxy gate* accepts the patch as soon as the visible target test passes. By construction, a both-miss patch has real completion 0: it fails some held-out behavior. The *Telos gate* runs held-out execution, rejects the patch, gives the generic generalization signal, lets the agent revise, and then re-runs the tests. A completion counts only if the final patch passes the target test and the held-out behavior.

On 7 `django` both-miss starts, the Telos gate lifts real completion from 0/7 to 5/7. For example, the `str(obj.pk)` hack is repaired, from the generic signal and the public bug description alone, to `str(getattr(obj, to_field_name))`, which is the correct general behavior. The result replicates on a disjoint older `django` sample: again 0/7 $\rightarrow$ 5/7. Pooled across the two samples, the gate moves real completion from 0/13 to 10/13.

The intervention also scales beyond `django` on the official SWE-bench harness. Across 20 both-miss starts from eight repositories, the proxy gate resolves 0/20 and the Telos gate recovers 10/20 to full official resolution. This is not a model leaderboard score: the baseline is definitional, and the oracle here is the repository regression suite, like a realistic continuous-integration gate. The claim is narrower and useful: rejection plus a generic generalization signal can convert some gamed patches into real fixes across repositories.

The gate preserves correct completions. Across 10 instances the gate accepts all 10 gold fixes and both genuinely-correct honest agent fixes with 0/12 false rejections. With the improvement result, the protocol is a net-positive filter: it converts gamed completions and leaves correct ones untouched. For the execution gate this control is confirmatory (a correct completion passes execution by construction); the gold-free property layer’s independent zero false-positive rate on correct code is a separate result in the program.

7 The intervention is oracle-agnostic

The sharpest objection is that the gate above used held-out tests. Those tests are realistic: they are the project’s own regression suite, the kind of check a software team would run in continuous integration. But they still contain information about the broken behavior. To test whether the method depends on that privileged test coverage, we replace the regression oracle with a gold-free one.

Gold-free means no hidden test and no reference patch enters the gate. A model reads the function’s documentation and the visible test, then proposes a property that should hold for correct code. For a formatter/parser pair, the property might be a round trip: formatting a value and parsing it should return the original value. For a mathematical function, the property might be an identity. The proposal is not trusted. It must pass the visible-test anchor and then be checked by execution on new inputs.

Pooled across runs on 7 evaluated instances, the gold-free property gate catches the gamed completion 7/7 and drives repair to real completion 5/7. That is close to the regression-gated 10/13 rate. The lesson is not that properties work everywhere; they do not. The lesson is that when a property can be derived, the intervention comes from execution-based rejection plus the generalization signal, not from secret access to the hidden tests.

A correction on the record. An initial gold-free run reported every property unsound and the oracle failing. This was a harness artifact, not a finding: the model’s standalone property scripts did not configure the framework’s settings, so instantiating objects raised a configuration error, which the model’s bare exception handler masked as a property failure. With the harness fixed, properties are sound on 9/10 of targeted functions and the result above holds. We keep the false-negative run and its cause on the record rather than discarding it silently.

8 The transferable insight

Across the program the same model has two very different roles.

As a verdict-giver, the model is asked, “Is this patch correct?” That is an ungrounded judgment, and the both-miss examples show that plausible wrong code can fool it. As a property-generator, the model is asked a different question: “What relation should correct code satisfy?” The answer becomes an artifact. The protocol then runs that artifact. If the property is bad, it fails the visible anchor or falsely rejects the known-correct code. If the patch is bad, execution falsifies it on new inputs.

This is the transferable idea: do not ask a model to be the source of truth when the claim can be converted into something executable. Let the model propose checks; let execution decide.

9 Limitations

We state the empirical bounds plainly.

- **Small N .** The replicated `django` intervention is pooled $N = 13$; the official cross-repository intervention is $N = 20$; the reward-model gaming measurement is $N = 20$; the gold-free property-gate rate is $N = 7$. These are pilots, not distributional results, and the confidence intervals are wide.

- **Constructed hacks.** The both-miss patches are adversarially constructed from gold fixes. This isolates the failure class, but it does not measure how often a naturally trained RL agent would discover the same class.
- **Repository coverage.** The latest intervention and reward-model measurements span eight repositories, but the pool is still small and selected for reward-hack-prone starts. Per-repository recovery varies, including zeros.
- **Single agent model.** The gaming agent and the repairing agent are the same frontier model; cross-model generalization of the intervention is untested.
- **Property-derivable subset.** The gold-free third layer applies where a function admits a checkable contract; a structural criterion is met by 0.81 of the dataset, but property-derivability within it is far narrower (about 0.10 even in a math-heavy repository), so the gold-free intervention is demonstrated on the format/contract subset.
- **No baseline comparison.** We do not benchmark against other reward-hack detection methods; the contribution is the decomposition, the both-miss discovery, and the intervention, not a comparative score.
- **Not a resolved-rate.** No result is a SWE-bench resolved-rate or a leaderboard number.

10 Conclusion

Static verification of an autonomous agent’s patch—by rules or by a frontier model—cannot reliably separate a correct general fix from a plausible special-case. Held-out execution is the recovering layer. A frontier adversary can manufacture both-miss patches, some survive multiple frontier judges, and in the cross-repository reward-model measurement one quarter survive every static verifier tested. Used as a gate, the same execution layer can also improve the agent’s work: it rejects the gamed patch, gives only a generic generalization signal, and often converts the patch into a real fix.

The central lesson is operational. A model is unsafe as the final verdict on code correctness, but useful as a generator of executable checks. The protocol becomes stronger when the model’s output is not trusted but run.

Reproducibility

Every quantitative claim in this paper is reproduced by a committed runner and a validated receipt in the Telos repository, under `experiments/iterNNN_*` with a pre-registered hypothesis, proof artifacts, and a learning record. The manuscript consolidates the public evidence arc through `iter152`. The deterministic parts rerun in continuous integration; native-execution and Docker transcripts are retained as observed evidence; the 20 saved reward-hack diffs from `iter152` are the seed for the next benchmark-building step.

References

- [1] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete Problems in AI Safety. *arXiv:1606.06565*, 2016.

- [2] V. Krakovna et al. Specification gaming: the flip side of AI ingenuity. DeepMind Blog, 2020.
- [3] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and Characterizing Reward Hacking. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. *arXiv:2306.05685*.
- [5] T. Y. Chen et al. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys*, 51(1), 2018.
- [6] K. Claessen and J. Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *Proc. ICFP*, 2000.
- [7] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *Proc. ICLR*, 2024. *arXiv:2310.06770*.